

```

//+-----
//|                                     RectangleZoneAnalyzer_05.mq4
//|
//|  Блок 05: хронологическая проверка паттернов из рейтинга.
//|
//|  Вход : touches.csv
//|          pattern_ranking.csv
//|
//|  Выход: pattern_walkforward.csv
//|          pattern_walkforward_summary.csv
//|
//|  История делится на равные календарные отрезки. На каждом
//|  отрезке каждый паттерн сравнивается со своей базой AGE_ONLY:
//|  тот же символ, ТФ, тип зоны и возрастная группа.
//+-----
#property copyright "Copyright 2026"
#property version  "1.00"
#property strict
#property script_show_inputs

//--- Файлы находятся в MQL4\Files
input string InputTouchesFileName           = "touches.csv";
input string InputRankingFileName           =
"pattern_ranking.csv";
input string OutputWalkForwardFileName      =
"pattern_walkforward.csv";
input string OutputWalkForwardSummaryFileName =
"pattern_walkforward_summary.csv";

//--- Обычно оба файла относятся к одному графику
input bool   FilterCurrentChartOnly         = true;

//--- Число равных календарных сегментов
input int    NumberOfSegments               = 6;

//--- Минимальная наполненность отдельного сегмента
input int    MinSegmentSamples              = 20;

```

```

input int      MinLocalDecisiveSamples          = 15;
input int      MinStructuralDecisiveSamples     = 15;

//--- Минимальное преимущество паттерна над AGE_ONLY в сегменте
input double   MinSegmentImprovementPctPoints  = 0.0;

//--- Условия итоговой устойчивости
input int      MinEligibleSegments              = 3;
input double   StablePassShare                  = 0.67;
input double   PromisingPassShare               = 0.50;
input bool     RequireLastSegmentPassForStable  = true;

//--- Границы торговых сессий должны совпадать с блоком 03
input int      AsiaEndHour                      = 7;
input int      LondonEndHour                    = 13;
input int      NewYorkEndHour                   = 21;

//--- Печать прогресса; 0 отключает
input int      ProgressEveryRows                = 5000;

#define TOUCHES_CSV_COLUMNS 137
#define RANKING_CSV_COLUMNS 41

//+-----

//| Описание принятого паттерна

//+-----
struct PatternDef
{
    int      rank;
    string   patternID;
    string   parentPatternID;
    string   selectionReason;
    string   reliabilityTier;
    string   patternType;
    string   ageBasis;
    string   symbol;
    string   timeframe;
    string   zoneType;
    string   ageBucket;
    string   touchBucket;
    string   depthBucket;
    string   approachBucket;
    string   heightBucket;
    string   sessionBucket;
    int      originalSamples;
    double   originalScore;
};

```

```
//+-----  
//| Статистика одного паттерна в одном сегменте
```

```
//+-----  
struct SegmentStat  
{  
    int samples;  
  
    int localReversal;  
    int localBreakout;  
  
    int structuralOpposite;  
    int structuralSameType;  
  
    double sumMFE;  
    int countMFE;  
    double sumMAE;  
    int countMAE;  
};
```

```
PatternDef g_patterns[];  
SegmentStat g_patternStats[];  
SegmentStat g_parentStats[];
```

```
datetime g_firstTouchTime = 0;  
datetime g_lastTouchTime = 0;  
datetime g_segmentStart[];  
datetime g_segmentEnd[];
```

```
//--- Индексы touches.csv  
int t_idxSymbol = -1;  
int t_idxTimeframe = -1;  
int t_idxZoneType = -1;  
int t_idxTouchNumber = -1;  
int t_idxTouchStatus = -1;  
int t_idxEntryDirection = -1;  
int t_idxDecisionTime = -1;  
int t_idxAgeCalendarMinutes = -1;  
int t_idxAgeTradingHours = -1;  
int t_idxZoneHeightATR = -1;  
int t_idxDepthPercent = -1;  
int t_idxApproachNetATR5 = -1;  
int t_idxLocalFirstResult = -1;  
int t_idxStructuralResult = -1;  
int t_idxMFE_CloseATR = -1;  
int t_idxMAE_CloseATR = -1;
```

```
//--- Индексы pattern_ranking.csv  
int r_idxRank = -1;  
int r_idxStatus = -1;
```

```

int r_idxSelectionReason      = -1;
int r_idxReliabilityTier     = -1;
int r_idxPatternID           = -1;
int r_idxParentPatternID     = -1;
int r_idxPatternType         = -1;
int r_idxAgeBasis            = -1;
int r_idxSymbol              = -1;
int r_idxTimeframe           = -1;
int r_idxZoneType            = -1;
int r_idxAgeBucket           = -1;
int r_idxTouchBucket         = -1;
int r_idxDepthBucket         = -1;
int r_idxApproachBucket      = -1;
int r_idxHeightBucket        = -1;
int r_idxSessionBucket       = -1;
int r_idxSamples             = -1;
int r_idxFinalScore          = -1;

int g_rowsScannedFirstPass   = 0;
int g_rowsScannedSecondPass  = 0;
int g_rowsIncluded           = 0;
int g_rowsSkipped            = 0;

```

//+-----

//| Имя текущего таймфрейма

//+-----

```

string TimeframeToString(int timeframe)
{
    switch(timeframe)
    {
        case PERIOD_M1:    return("M1");
        case PERIOD_M5:    return("M5");
        case PERIOD_M15:   return("M15");
        case PERIOD_M30:   return("M30");
        case PERIOD_H1:    return("H1");
        case PERIOD_H4:    return("H4");
        case PERIOD_D1:    return("D1");
        case PERIOD_W1:    return("W1");
        case PERIOD_MN1:   return("MN1");
    }

    return(IntegerToString(timeframe));
}

```

//+-----

//| Удаление UTF-8 ВОМ из первого заголовка

```
//+-----
string StripBOM(string value)
{
    if(StringLen(value) > 0 && StringGetCharacter(value, 0) ==
65279)
        return(StringSubstr(value, 1));

    // Возможное представление UTF-8 BOM в ANSI-чтении
    if(StringLen(value) >= 3 &&
        StringGetCharacter(value, 0) == 239 &&
        StringGetCharacter(value, 1) == 187 &&
        StringGetCharacter(value, 2) == 191)
    {
        return(StringSubstr(value, 3));
    }

    return(value);
}
```

```
//+-----
//| Поиск колонки
```

```
//+-----
int FindColumn(string &headers[], string columnName)
{
    int count = ArraySize(headers);

    for(int i = 0; i < count; i++)
    {
        string current = headers[i];

        if(i == 0)
            current = StripBOM(current);

        if(current == columnName)
            return(i);
    }

    return(-1);
}
```

```
//+-----
//| Чтение фиксированного количества CSV-полей
```

```
//+-----
bool ReadCsvRecord(int handle, int columns, string &fields[])
{

```

```

    ArrayResize(fields, columns);

    if(FileIsEnding(handle))
        return(false);

    for(int i = 0; i < columns; i++)
    {
        if(FileIsEnding(handle) && i == 0)
            return(false);

        fields[i] = FileReadString(handle);
    }

    return(StringLen(fields[0]) > 0);
}

//+-----
//| Преобразования
//+-----
int ParseInt(string value, int defaultValue = 0)
{
    if(StringLen(value) == 0)
        return(defaultValue);

    return((int)StringToInteger(value));
}

double ParseDouble(string value, double defaultValue = 0.0)
{
    if(StringLen(value) == 0)
        return(defaultValue);

    return(StringToDouble(value));
}

//+-----
//| Карта колонок ranking
//+-----
bool MapRankingColumns(string &headers[])
{
    r_idxRank          = FindColumn(headers, "Rank");
    r_idxStatus        = FindColumn(headers, "Status");
    r_idxSelectionReason = FindColumn(headers, "SelectionReason"
);
    r_idxReliabilityTier = FindColumn(headers, "ReliabilityTier"
);
}

```

```

    r_idxPatternID      = FindColumn(headers, "PatternID");
    r_idxParentPatternID = FindColumn(headers, "ParentPatternID"
);
    r_idxPatternType     = FindColumn(headers, "PatternType");
    r_idxAgeBasis        = FindColumn(headers, "AgeBasis");
    r_idxSymbol          = FindColumn(headers, "Symbol");
    r_idxTimeframe       = FindColumn(headers, "Timeframe");
    r_idxZoneType        = FindColumn(headers, "ZoneType");
    r_idxAgeBucket       = FindColumn(headers, "AgeBucket");
    r_idxTouchBucket     = FindColumn(headers, "TouchBucket");
    r_idxDepthBucket     = FindColumn(headers, "DepthBucket");
    r_idxApproachBucket  = FindColumn(headers, "ApproachBucket"
);
    r_idxHeightBucket    = FindColumn(headers,
"ZoneHeightBucket");
    r_idxSessionBucket   = FindColumn(headers, "SessionBucket");
    r_idxSamples         = FindColumn(headers, "Samples");
    r_idxFinalScore      = FindColumn(headers, "FinalScore");

    if(r_idxRank < 0 || r_idxStatus < 0 ||
        r_idxSelectionReason < 0 || r_idxReliabilityTier < 0 ||
        r_idxPatternID < 0 || r_idxParentPatternID < 0 ||
        r_idxPatternType < 0 || r_idxAgeBasis < 0 ||
        r_idxSymbol < 0 || r_idxTimeframe < 0 ||
        r_idxZoneType < 0 || r_idxAgeBucket < 0 ||
        r_idxTouchBucket < 0 || r_idxDepthBucket < 0 ||
        r_idxApproachBucket < 0 || r_idxHeightBucket < 0 ||
        r_idxSessionBucket < 0 || r_idxSamples < 0 ||
        r_idxFinalScore < 0)
    {
        Print(
"Ошибка: pattern_ranking.csv не содержит обязательных колонок."
);
        return(false);
    }

    return(true);
}

//+-----

//| Карта колонок touches

//+-----
bool MapTouchesColumns(string &headers[])
{
    t_idxSymbol          = FindColumn(headers, "Symbol");
    t_idxTimeframe       = FindColumn(headers, "Timeframe");
    t_idxZoneType        = FindColumn(headers, "ZoneType");
    t_idxTouchNumber     = FindColumn(headers, "TouchNumber"
);

```

```

    t_idxTouchStatus          = FindColumn(headers, "TouchStatus"
);
    t_idxEntryDirection       = FindColumn(headers,
"EntryDirection");
    t_idxDecisionTime         = FindColumn(headers,
"TouchDecisionTime");
    t_idxAgeCalendarMinutes   = FindColumn(headers,
"ZoneAgeCalendarMinutes");
    t_idxAgeTradingHours      = FindColumn(headers,
"ZoneAgeTradingHours");
    t_idxZoneHeightATR        = FindColumn(headers,
"ZoneHeightATR");
    t_idxDepthPercent         = FindColumn(headers,
"TouchDepthPercent");
    t_idxApproachNetATR5      = FindColumn(headers,
"ApproachNetMoveATR_5");
    t_idxLocalFirstResult     = FindColumn(headers,
"LocalFirstResult");
    t_idxStructuralResult     = FindColumn(headers,
"StructuralFirstResult");
    t_idxMFE_CloseATR         = FindColumn(headers,
"MFE_CloseATR");
    t_idxMAE_CloseATR         = FindColumn(headers,
"MAE_CloseATR");

    if(t_idxSymbol < 0 || t_idxTimeframe < 0 || t_idxZoneType <
0 ||
        t_idxTouchNumber < 0 || t_idxTouchStatus < 0 ||
        t_idxEntryDirection < 0 || t_idxDecisionTime < 0 ||
        t_idxAgeCalendarMinutes < 0 || t_idxAgeTradingHours < 0
    ||
        t_idxZoneHeightATR < 0 || t_idxDepthPercent < 0 ||
        t_idxApproachNetATR5 < 0 || t_idxLocalFirstResult < 0 ||
        t_idxStructuralResult < 0 || t_idxMFE_CloseATR < 0 ||
        t_idxMAE_CloseATR < 0)
    {
        Print(
"Ошибка: touches.csv не содержит обязательных колонок.");
        return(false);
    }

    return(true);
}

//+-----

//| Загрузка паттернов рейтинга

//+-----
bool LoadPatterns()
{

```

```

ResetLastError();

int handle = FileOpen(
    InputRankingFileName,
    FILE_READ | FILE_CSV | FILE_ANSI,
    ';'
);

if(handle == INVALID_HANDLE)
{
    Print("Ошибка открытия ", InputRankingFileName,
        ". Код ошибки: ", GetLastError());
    ResetLastError();
    return(false);
}

string headers[];
ArrayResize(headers, RANKING_CSV_COLUMNS);

for(int i = 0; i < RANKING_CSV_COLUMNS; i++)
    headers[i] = FileReadString(handle);

if(!MapRankingColumns(headers))
{
    FileClose(handle);
    return(false);
}

ArrayResize(g_patterns, 0);
string fields[];
int readRows = 0;

while(ReadCsvRecord(handle, RANKING_CSV_COLUMNS, fields))
{
    readRows++;

    if(fields[r_idxStatus] != "ACCEPTED")
        continue;

    if(FilterCurrentChartOnly)
    {
        if(fields[r_idxSymbol] != Symbol() ||
            fields[r_idxTimeframe] != TimeframeToString(Period
)))
        {
            continue;
        }
    }

    int size = ArraySize(g_patterns);

    if(ArrayResize(g_patterns, size + 1) != size + 1)

```

```

    {
        Print("Ошибка памяти при загрузке паттернов.");
        FileClose(handle);
        return(false);
    }

    g_patterns[size].rank = ParseInt(fields[
r_idxRank]);
    g_patterns[size].patternID = fields[r_idxPatternID
];
    g_patterns[size].parentPatternID = fields[
r_idxParentPatternID];
    g_patterns[size].selectionReason = fields[
r_idxSelectionReason];
    g_patterns[size].reliabilityTier = fields[
r_idxReliabilityTier];
    g_patterns[size].patternType = fields[
r_idxPatternType];
    g_patterns[size].ageBasis = fields[r_idxAgeBasis
];
    g_patterns[size].symbol = fields[r_idxSymbol];
    g_patterns[size].timeframe = fields[r_idxTimeframe
];
    g_patterns[size].zoneType = fields[r_idxZoneType
];
    g_patterns[size].ageBucket = fields[r_idxAgeBucket
];
    g_patterns[size].touchBucket = fields[
r_idxTouchBucket];
    g_patterns[size].depthBucket = fields[
r_idxDepthBucket];
    g_patterns[size].approachBucket = fields[
r_idxApproachBucket];
    g_patterns[size].heightBucket = fields[
r_idxHeightBucket];
    g_patterns[size].sessionBucket = fields[
r_idxSessionBucket];
    g_patterns[size].originalSamples = ParseInt(fields[
r_idxSamples]);
    g_patterns[size].originalScore = ParseDouble(fields[
r_idxFinalScore]);
}

FileClose(handle);

Print("Блок 05: прочитано строк рейтинга: ", readRows,
      ", принято паттернов: ", ArraySize(g_patterns));

if(ArraySize(g_patterns) == 0)
{
    Print(
"Ошибка: в рейтинге нет подходящих АСЕРТЕД-паттернов.");
}

```

```

        return(false);
    }

    return(true);
}

//+-----
//| Проверка строки касания для анализа
//+-----
bool IsIncludedTouch(string &fields[])
{
    if(fields[t_idxTouchStatus] != "TOUCH" ||
        fields[t_idxEntryDirection] != "EXPECTED_SIDE")
    {
        return(false);
    }

    if(FilterCurrentChartOnly)
    {
        if(fields[t_idxSymbol] != Symbol() ||
            fields[t_idxTimeframe] != TimeframeToString(Period()))
        {
            return(false);
        }
    }

    if(StringLen(fields[t_idxDecisionTime]) == 0)
        return(false);

    return(true);
}

//+-----
//| Первый проход: границы истории
//+-----
bool ScanTouchTimeRange()
{
    ResetLastError();

    int handle = FileOpen(
        InputTouchesFileName,
        FILE_READ | FILE_CSV | FILE_ANSI,
        ';'
    );

    if(handle == INVALID_HANDLE)

```

```

{
    Print("Ошибка открытия ", InputTouchesFileName,
        ". Код ошибки: ", GetLastError());
    ResetLastError();
    return(false);
}

string headers[];
ArrayResize(headers, TOUCHES_CSV_COLUMNS);

for(int i = 0; i < TOUCHES_CSV_COLUMNS; i++)
    headers[i] = FileReadString(handle);

if(!MapTouchesColumns(headers))
{
    FileClose(handle);
    return(false);
}

g_firstTouchTime = 0;
g_lastTouchTime = 0;
g_rowsScannedFirstPass = 0;

string fields[];

while(ReadCsvRecord(handle, TOUCHES_CSV_COLUMNS, fields))
{
    g_rowsScannedFirstPass++;

    if(!IsIncludedTouch(fields))
        continue;

    datetime eventTime = StringToTime(fields[
t_idxDecisionTime]);

    if(eventTime <= 0)
        continue;

    if(g_firstTouchTime == 0 || eventTime < g_firstTouchTime)
        g_firstTouchTime = eventTime;

    if(g_lastTouchTime == 0 || eventTime > g_lastTouchTime)
        g_lastTouchTime = eventTime;

    if(ProgressEveryRows > 0 &&
        g_rowsScannedFirstPass % ProgressEveryRows == 0)
    {
        Print("Блок 05, проход 1: прочитано строк ",
            g_rowsScannedFirstPass);
    }
}

```

```

    FileClose(handle);

    if(g_firstTouchTime <= 0 || g_lastTouchTime <
g_firstTouchTime)
    {
        Print("Ошибка: не найден диапазон правильных касаний.");
        return(false);
    }

    Print("Блок 05: диапазон касаний: ",
        TimeToString(g_firstTouchTime, TIME_DATE |
TIME_MINUTES),
        " - ",
        TimeToString(g_lastTouchTime, TIME_DATE | TIME_MINUTES
));

    return(true);
}

//+-----

//| Построение сегментов

//+-----
bool BuildSegments()
{
    if(NumberOfSegments < 2 || NumberOfSegments > 50)
    {
        Print("Ошибка: NumberOfSegments должен быть от 2 до 50."
);
        return(false);
    }

    long totalSeconds = (long) (g_lastTouchTime -
g_firstTouchTime) + 1;

    if(totalSeconds < NumberOfSegments)
    {
        Print(
"Ошибка: диапазон истории слишком мал для выбранного числа сегментов
");
        return(false);
    }

    ArrayResize(g_segmentStart, NumberOfSegments);
    ArrayResize(g_segmentEnd, NumberOfSegments);

    for(int segment = 0; segment < NumberOfSegments; segment++)
    {
        long startOffset = (totalSeconds * segment) /
NumberOfSegments;

```

```

        long endOffset    = (totalSeconds * (segment + 1)) /
NumberOfSegments - 1;

        g_segmentStart[segment] =
            (datetime)((long)g_firstTouchTime + startOffset);

        g_segmentEnd[segment] =
            (datetime)((long)g_firstTouchTime + endOffset);
    }

    g_segmentEnd[NumberOfSegments - 1] = g_lastTouchTime;

    return(true);
}

//+-----
//| Индекс сегмента
//+-----
int GetSegmentIndex(datetime eventTime)
{
    if(eventTime <= g_firstTouchTime)
        return(0);

    if(eventTime >= g_lastTouchTime)
        return(NumberOfSegments - 1);

    long totalSeconds = (long)(g_lastTouchTime -
g_firstTouchTime) + 1;
    long offset      = (long)(eventTime - g_firstTouchTime);

    int index = (int)((offset * NumberOfSegments) / totalSeconds
);

    if(index < 0)
        index = 0;

    if(index >= NumberOfSegments)
        index = NumberOfSegments - 1;

    return(index);
}

//+-----
//| Сброс статистики
//+-----
void ResetSegmentStat(SegmentStat &stat)

```

```

{
    stat.samples = 0;
    stat.localReversal = 0;
    stat.localBreakout = 0;
    stat.structuralOpposite = 0;
    stat.structuralSameType = 0;
    stat.sumMFE = 0.0;
    stat.countMFE = 0;
    stat.sumMAE = 0.0;
    stat.countMAE = 0;
}

//+-----

//| Выделение массивов статистики

//+-----
bool AllocateStatistics()
{
    int total = ArraySize(g_patterns) * NumberOfSegments;

    if(ArrayResize(g_patternStats, total) != total ||
       ArrayResize(g_parentStats, total) != total)
    {
        Print("Ошибка памяти при создании массивов статистики.");
        return(false);
    }

    for(int i = 0; i < total; i++)
    {
        ResetSegmentStat(g_patternStats[i]);
        ResetSegmentStat(g_parentStats[i]);
    }

    return(true);
}

int StatIndex(int patternIndex, int segmentIndex)
{
    return(patternIndex * NumberOfSegments + segmentIndex);
}

//+-----

//| Категории, полностью совпадающие с блоком 03

//+-----
string GetAgeBucketName(double ageMinutes)
{
    if(ageMinutes < 15.0)    return("00_0_15_MIN");
}

```

```

    if (ageMinutes < 30.0)    return("01_15_30_MIN");
    if (ageMinutes < 60.0)    return("02_30_60_MIN");
    if (ageMinutes < 120.0)   return("03_1_2_HOURS");
    if (ageMinutes < 240.0)   return("04_2_4_HOURS");
    if (ageMinutes < 480.0)   return("05_4_8_HOURS");
    if (ageMinutes < 1440.0)  return("06_8_24_HOURS");
    if (ageMinutes < 2880.0)  return("07_24_48_HOURS");
    return("08_48_HOURS_PLUS");
}

string GetTouchBucketName(int touchNumber)
{
    if (touchNumber <= 1) return("TOUCH_1");
    if (touchNumber == 2) return("TOUCH_2");
    if (touchNumber == 3) return("TOUCH_3");
    if (touchNumber == 4) return("TOUCH_4");
    return("TOUCH_5_PLUS");
}

string GetDepthBucketName(double depthPercent)
{
    if (depthPercent < 25.0) return("DEPTH_0_25_PCT");
    if (depthPercent < 50.0) return("DEPTH_25_50_PCT");
    if (depthPercent < 75.0) return("DEPTH_50_75_PCT");
    return("DEPTH_75_100_PCT");
}

string GetApproachBucketName(double approachNetATR5)
{
    if (approachNetATR5 <= 0.0) return("APPROACH_FLAT_OR_AWAY");
    if (approachNetATR5 < 0.5)  return("APPROACH_SLOW_LT_0_5_ATR");
};
    if (approachNetATR5 < 1.0)  return(
"APPROACH_MEDIUM_0_5_1_ATR");
    return("APPROACH_FAST_GE_1_ATR");
}

string GetHeightBucketName(double zoneHeightATR)
{
    if (zoneHeightATR < 0.25) return("HEIGHT_LT_0_25_ATR");
    if (zoneHeightATR < 0.50) return("HEIGHT_0_25_0_50_ATR");
    if (zoneHeightATR < 0.75) return("HEIGHT_0_50_0_75_ATR");
    if (zoneHeightATR < 1.00) return("HEIGHT_0_75_1_00_ATR");
    return("HEIGHT_GE_1_00_ATR");
}

string GetSessionBucketName(datetime decisionTime)
{
    int hour = TimeHour(decisionTime);

    if (hour < AsiaEndHour)
        return("ASIA_SERVER_TIME");
}

```

```

    if(hour < LondonEndHour)
        return("LONDON_SERVER_TIME");

    if(hour < NewYorkEndHour)
        return("NEW_YORK_SERVER_TIME");

    return("OTHER_SERVER_TIME");
}

```

```

//+-----
//| Совпадение касания с базой AGE_ONLY

```

```

//+-----
bool MatchesParent(
    PatternDef &pattern,
    string symbol,
    string timeframe,
    string zoneType,
    string ageBucket
)
{
    return(
        symbol == pattern.symbol &&
        timeframe == pattern.timeframe &&
        zoneType == pattern.zoneType &&
        ageBucket == pattern.ageBucket
    );
}

```

```

//+-----
//| Совпадение касания с полным паттерном

```

```

//+-----
bool MatchesPattern(
    PatternDef &pattern,
    string symbol,
    string timeframe,
    string zoneType,
    string ageBucket,
    string touchBucket,
    string depthBucket,
    string approachBucket,
    string heightBucket,
    string sessionBucket
)
{
    if(!MatchesParent(pattern, symbol, timeframe, zoneType,
ageBucket))

```

```

        return(false);

        if(pattern.touchBucket != "ALL" && pattern.touchBucket !=
touchBucket)
            return(false);

        if(pattern.depthBucket != "ALL" && pattern.depthBucket !=
depthBucket)
            return(false);

        if(pattern.approachBucket != "ALL" &&
pattern.approachBucket != approachBucket)
        {
            return(false);
        }

        if(pattern.heightBucket != "ALL" && pattern.heightBucket !=
heightBucket)
            return(false);

        if(pattern.sessionBucket != "ALL" && pattern.sessionBucket
!= sessionBucket)
            return(false);

        return(true);
    }

```

```

//+-----
//| Добавление результата касания

```

```

//+-----
void UpdateStat(SegmentStat &stat, string &fields[])
{
    stat.samples++;

    string localResult = fields[t_idxLocalFirstResult];

    if(localResult == "REVERSAL_FIRST")
        stat.localReversal++;
    else if(localResult == "BREAKOUT_FIRST")
        stat.localBreakout++;

    string structuralResult = fields[t_idxStructuralResult];

    if(structuralResult == "OPPOSITE_ZONE_FIRST")
        stat.structuralOpposite++;
    else if(structuralResult == "SAME_TYPE_ZONE_FIRST")
        stat.structuralSameType++;

    if(StringLen(fields[t_idxMFE_CloseATR]) > 0)

```

```

{
    stat.sumMFE += StringToDouble(fields[t_idxMFE_CloseATR]);
    stat.countMFE++;
}

if(StringLen(fields[t_idxMAE_CloseATR]) > 0)
{
    stat.sumMAE += StringToDouble(fields[t_idxMAE_CloseATR]);
    stat.countMAE++;
}
}

//+-----+
//| Второй проход: накопление статистики
//+-----+
bool AggregateSegments()
{
    ResetLastError();

    int handle = FileOpen(
        InputTouchesFileName,
        FILE_READ | FILE_CSV | FILE_ANSI,
        ';'
    );

    if(handle == INVALID_HANDLE)
    {
        Print("Ошибка повторного открытия ", InputTouchesFileName
        ,
            ". Код ошибки: ", GetLastError());
        ResetLastError();
        return(false);
    }

    string headers[];
    ArrayResize(headers, TOUCHES_CSV_COLUMNS);

    for(int i = 0; i < TOUCHES_CSV_COLUMNS; i++)
        headers[i] = FileReadString(handle);

    if(!MapTouchesColumns(headers))
    {
        FileClose(handle);
        return(false);
    }

    string fields[];
    g_rowsScannedSecondPass = 0;
    g_rowsIncluded = 0;

```

```

g_rowsSkipped = 0;

int patternCount = ArraySize(g_patterns);

while(ReadCsvRecord(handle, TOUCHES_CSV_COLUMNS, fields))
{
    g_rowsScannedSecondPass++;

    if(!IsIncludedTouch(fields))
    {
        g_rowsSkipped++;
        continue;
    }

    datetime decisionTime = StringToTime(fields[
t_idxDecisionTime]);

    if(decisionTime <= 0)
    {
        g_rowsSkipped++;
        continue;
    }

    int touchNumber = ParseInt(fields[t_idxTouchNumber]);
    double calendarAgeMinutes =
        ParseDouble(fields[t_idxAgeCalendarMinutes], -1.0);
    double tradingAgeMinutes =
        ParseDouble(fields[t_idxAgeTradingHours], -1.0) * 60.0
;

    double depthPercent =
        ParseDouble(fields[t_idxDepthPercent], -1.0);
    double approachNetATR5 =
        ParseDouble(fields[t_idxApproachNetATR5], 0.0);
    double zoneHeightATR =
        ParseDouble(fields[t_idxZoneHeightATR], -1.0);

    if(touchNumber < 1 || calendarAgeMinutes < 0.0 ||
        tradingAgeMinutes < 0.0 || depthPercent < 0.0 ||
        depthPercent > 100.0001 || zoneHeightATR < 0.0)
    {
        g_rowsSkipped++;
        continue;
    }

    string symbol = fields[t_idxSymbol];
    string timeframe = fields[t_idxTimeframe];
    string zoneType = fields[t_idxZoneType];
    string touchBucket = GetTouchBucketName(touchNumber);
    string depthBucket = GetDepthBucketName(depthPercent);
    string approachBucket = GetApproachBucketName(
approachNetATR5);
    string heightBucket = GetHeightBucketName(zoneHeightATR);

```

```

);

string sessionBucket = GetSessionBucketName(decisionTime);

int segment = GetSegmentIndex(decisionTime);
bool matchedAny = false;

for(int p = 0; p < patternCount; p++)
{
    double ageMinutes;

    if(g_patterns[p].ageBasis == "CALENDAR_TIME")
        ageMinutes = calendarAgeMinutes;
    else
        ageMinutes = tradingAgeMinutes;

    string ageBucket = GetAgeBucketName(ageMinutes);

    if(!MatchesParent(
        g_patterns[p],
        symbol,
        timeframe,
        zoneType,
        ageBucket
    ))
    {
        continue;
    }

    int index = StatIndex(p, segment);
    UpdateStat(g_parentStats[index], fields);

    if(MatchesPattern(
        g_patterns[p],
        symbol,
        timeframe,
        zoneType,
        ageBucket,
        touchBucket,
        depthBucket,
        approachBucket,
        heightBucket,
        sessionBucket
    ))
    {
        UpdateStat(g_patternStats[index], fields);
        matchedAny = true;
    }
}

if(matchedAny)
    g_rowsIncluded++;

```

```

        if(ProgressEveryRows > 0 &&
           g_rowsScannedSecondPass % ProgressEveryRows == 0)
        {
            Print("Блок 05, проход 2: прочитано строк ",
                  g_rowsScannedSecondPass);
        }
    }

    FileClose(handle);

    Print("Блок 05: строк касаний прочитано: ",
          g_rowsScannedSecondPass,
          ", совпало хотя бы с одним паттерном: ",
          g_rowsIncluded,
          ", пропущено: ", g_rowsSkipped);

    return(true);
}

//+-----

//| Статистические функции

//+-----

double Percent(int successes, int total)
{
    if(total <= 0)
        return(-1.0);

    return(100.0 * successes / total);
}

double WilsonLower95(int successes, int total)
{
    if(total <= 0)
        return(-1.0);

    double z = 1.959963984540054;
    double n = total;
    double p = successes / n;
    double z2 = z * z;

    double center = p + z2 / (2.0 * n);
    double margin = z * MathSqrt(
        (p * (1.0 - p) + z2 / (4.0 * n)) / n
    );
    double denominator = 1.0 + z2 / n;

    return(100.0 * (center - margin) / denominator);
}

```

```

double Average(double sum, int count)
{
    if(count <= 0)
        return(-1.0);

    return(sum / count);
}

double SafeRatio(double numerator, double denominator)
{
    if(numerator < 0.0 || denominator <= 0.0)
        return(-1.0);

    return(numerator / denominator);
}

//+-----
//| CSV-форматирование
//+-----
void AppendField(string &line, string value)
{
    if(StringLen(line) > 0)
        line += ";";

    line += value;
}

string DoubleOrBlank(double value, int digits)
{
    if(value < -0.5)
        return("");

    return(DoubleToString(value, digits));
}

string SignedDoubleOrBlank(double value, int digits, bool valid)
{
    if(!valid)
        return("");

    return(DoubleToString(value, digits));
}

string DateTimeText(datetime value)
{
    return(TimeToString(value, TIME_DATE | TIME_MINUTES));
}

```

```

string YesNo(bool value)
{
    return(value ? "YES" : "NO");
}

//+-----

//| Условия преимущества сегмента

//+-----
bool IsLocalEdgeRequired(PatternDef &pattern)
{
    return(
        pattern.selectionReason == "LOCAL_EDGE" ||
        pattern.selectionReason == "LOCAL_AND_STRUCTURAL_EDGE"
    );
}

bool IsStructuralEdgeRequired(PatternDef &pattern)
{
    return(
        pattern.selectionReason == "STRUCTURAL_EDGE" ||
        pattern.selectionReason == "LOCAL_AND_STRUCTURAL_EDGE"
    );
}

//+-----

//| Сохранение подробных сегментов

//+-----
bool SaveWalkForwardDetails()
{
    ResetLastError();

    int handle = FileOpen(
        OutputWalkForwardFileName,
        FILE_WRITE | FILE_TXT | FILE_ANSI
    );

    if(handle == INVALID_HANDLE)
    {
        Print("Ошибка создания ", OutputWalkForwardFileName,
            ". Код ошибки: ", GetLastError());
        ResetLastError();
        return(false);
    }

    string header =

```

```

"Rank;PatternID;SelectionReason;ReliabilityTier;PatternType;AgeBasis;
+
"Symbol;Timeframe;ZoneType;AgeBucket;TouchBucket;DepthBucket;"
+
"ApproachBucket;ZoneHeightBucket;SessionBucket;Segment;SegmentStart;
+
"SegmentEnd;Samples;LocalDecisive;LocalReversals;LocalReversalPct;"
+
    "LocalWilsonLower95;ParentSamples;ParentLocalDecisive;" +
    "ParentLocalReversalPct;ParentLocalWilsonLower95;" +
"LocalImprovementPctPoints;StructuralDecisive;StructuralOpposite;"
+
    "StructuralOppositePct;StructuralWilsonLower95;" +
    "ParentStructuralDecisive;ParentStructuralOppositePct;" +
"ParentStructuralWilsonLower95;StructuralImprovementPctPoints;"
+
    "AvgMFE_CloseATR;AvgMAE_CloseATR;MFE_MAE_Ratio;" +
"EligibleLocal;EligibleStructural;LocalEdge;StructuralEdge;SegmentPa
;

FileWriteString(handle, header);

int patternCount = ArraySize(g_patterns);

for(int p = 0; p < patternCount; p++)
{
    for(int s = 0; s < NumberOfSegments; s++)
    {
        int index = StatIndex(p, s);
        SegmentStat patternStat = g_patternStats[index];
        SegmentStat parentStat = g_parentStats[index];

        int localDecisive =
            patternStat.localReversal +
patternStat.localBreakout;
        int parentLocalDecisive =
            parentStat.localReversal + parentStat.localBreakout
;

        int structuralDecisive =
            patternStat.structuralOpposite +
patternStat.structuralSameType;
        int parentStructuralDecisive =
            parentStat.structuralOpposite +
parentStat.structuralSameType;

        double localPct = Percent15438
            patternStat.localReversal,
            parentStat.localReversal,
            patternStat.localBreakout,
            parentStat.localBreakout,
            100.0;
        double parentPct = Percent15438
            parentStat.localReversal,
            parentStat.localReversal,
            parentStat.localBreakout,
            parentStat.localBreakout,
            100.0;
    }
}

```

```

        Percent(parentStat.localReversal,
parentLocalDecisive);
        double parentLocalWilson =
            WilsonLower95(parentStat.localReversal,
parentLocalDecisive);

        double structuralPct =
            Percent(patternStat.structuralOpposite,
structuralDecisive);
        double structuralWilson =
            WilsonLower95(patternStat.structuralOpposite,
structuralDecisive);
        double parentStructuralPct =
            Percent(parentStat.structuralOpposite,
parentStructuralDecisive);
        double parentStructuralWilson =
            WilsonLower95(parentStat.structuralOpposite,
parentStructuralDecisive);

        double localImprovement = -1.0;
        double structuralImprovement = -1.0;

        if(localPct >= 0.0 && parentLocalPct >= 0.0)
            localImprovement = localPct - parentLocalPct;

        if(structuralPct >= 0.0 && parentStructuralPct >= 0.0)
            structuralImprovement = structuralPct -
parentStructuralPct;

        double avgMFE = Average(patternStat.sumMFE,
patternStat.countMFE);
        double avgMAE = Average(patternStat.sumMAE,
patternStat.countMAE);
        double ratio = SafeRatio(avgMFE, avgMAE);

        bool eligibleLocal =
            patternStat.samples >= MinSegmentSamples &&
            localDecisive >= MinLocalDecisiveSamples &&
            parentLocalDecisive >= MinLocalDecisiveSamples;

        bool eligibleStructural =
            patternStat.samples >= MinSegmentSamples &&
            structuralDecisive >= MinStructuralDecisiveSamples
&&
            parentStructuralDecisive >=
MinStructuralDecisiveSamples;

        bool localEdge =
            eligibleLocal &&
            localImprovement >= MinSegmentImprovementPctPoints;

        bool structuralEdge =

```

```

        eligibleStructural &&
        structuralImprovement >=
MinSegmentImprovementPctPoints;

    bool segmentPass = true;

    if(IsLocalEdgeRequired(g_patterns[p]))
        segmentPass = segmentPass && localEdge;

    if(IsStructuralEdgeRequired(g_patterns[p]))
        segmentPass = segmentPass && structuralEdge;

    string line = "";

    AppendField(line, IntegerToString(g_patterns[p].rank
));
    AppendField(line, g_patterns[p].patternID);
    AppendField(line, g_patterns[p].selectionReason);
    AppendField(line, g_patterns[p].reliabilityTier);
    AppendField(line, g_patterns[p].patternType);
    AppendField(line, g_patterns[p].ageBasis);
    AppendField(line, g_patterns[p].symbol);
    AppendField(line, g_patterns[p].timeframe);
    AppendField(line, g_patterns[p].zoneType);
    AppendField(line, g_patterns[p].ageBucket);
    AppendField(line, g_patterns[p].touchBucket);
    AppendField(line, g_patterns[p].depthBucket);
    AppendField(line, g_patterns[p].approachBucket);
    AppendField(line, g_patterns[p].heightBucket);
    AppendField(line, g_patterns[p].sessionBucket);
    AppendField(line, IntegerToString(s + 1));
    AppendField(line, DateTimeText(g_segmentStart[s]));
    AppendField(line, DateTimeText(g_segmentEnd[s]));
    AppendField(line, IntegerToString(patternStat.samples
));
    AppendField(line, IntegerToString(localDecisive));
    AppendField(line, IntegerToString(
patternStat.localReversal));
    AppendField(line, DoubleOrBlank(localPct, 4));
    AppendField(line, DoubleOrBlank(localWilson, 6));
    AppendField(line, IntegerToString(parentStat.samples
));
    AppendField(line, IntegerToString(parentLocalDecisive
));
    AppendField(line, DoubleOrBlank(parentLocalPct, 4));
    AppendField(line, DoubleOrBlank(parentLocalWilson, 6
));
    AppendField(
        line,
        SignedDoubleOrBlank(
            localImprovement,
            4,

```

```

        localPct >= 0.0 && parentLocalPct >= 0.0
    )
);
AppendField(line, IntegerToString(structuralDecisive
));
AppendField(line, IntegerToString(
patternStat.structuralOpposite));
AppendField(line, DoubleOrBlank(structuralPct, 4));
AppendField(line, DoubleOrBlank(structuralWilson, 6));
AppendField(line, IntegerToString(
parentStructuralDecisive));
AppendField(line, DoubleOrBlank(parentStructuralPct, 4
));
AppendField(line, DoubleOrBlank(parentStructuralWilson
, 6));
AppendField(
    line,
    SignedDoubleOrBlank(
        structuralImprovement,
        4,
        structuralPct >= 0.0 && parentStructuralPct >=
0.0
    )
);
AppendField(line, DoubleOrBlank(avgMFE, 6));
AppendField(line, DoubleOrBlank(avgMAE, 6));
AppendField(line, DoubleOrBlank(ratio, 6));
AppendField(line, YesNo(eligibleLocal));
AppendField(line, YesNo(eligibleStructural));
AppendField(line, YesNo(localEdge));
AppendField(line, YesNo(structuralEdge));
AppendField(line, YesNo(segmentPass));

FileWriteString(handle, line + "\r\n");
}
}

FileFlush(handle);
FileClose(handle);

Print("Файл создан: ",
    TerminalInfoString(TERMINAL_DATA_PATH),
    "\\MQL4\\Files\\", OutputWalkForwardFileName);

return(true);
}

```

//+-----

//| Сохранение итоговой устойчивости

```
//+-----
bool SaveWalkForwardSummary()
{
    ResetLastError();

    int handle = FileOpen(
        OutputWalkForwardSummaryFileName,
        FILE_WRITE | FILE_TXT | FILE_ANSI
    );

    if (handle == INVALID_HANDLE)
    {
        Print("Ошибка создания ",
OutputWalkForwardSummaryFileName,
        ". Код ошибки: ", GetLastError());
        ResetLastError();
        return(false);
    }

    string header =

"Rank;PatternID;SelectionReason;ReliabilityTier;PatternType;AgeBasis;
+
"Symbol;Timeframe;ZoneType;AgeBucket;TouchBucket;DepthBucket;"
+
"ApproachBucket;ZoneHeightBucket;SessionBucket;OriginalSamples;"
+
"OriginalFinalScore;TotalSegments;EligibleSegments;PassSegments;"
+
"PassSharePct;LocalEligibleSegments;LocalPositiveSegments;" +
    "LocalPositiveSharePct;AvgLocalImprovementPctPoints;" +
"WorstLocalImprovementPctPoints;BestLocalImprovementPctPoints;"
+
    "StructuralEligibleSegments;StructuralPositiveSegments;"
+
"StructuralPositiveSharePct;AvgStructuralImprovementPctPoints;"
+
"WorstStructuralImprovementPctPoints;BestStructuralImprovementPctPoints;"
+
    "LastSegmentSamples;LastLocalImprovementPctPoints;" +
    "LastStructuralImprovementPctPoints;LastSegmentPass;" +
    "StabilityGrade;StableAccepted\r\n";

    FileWriteString(handle, header);

    int stableCount = 0;
    Страница 29/38

```

```

for(int p = 0; p < patternCount; p++)
{
    int eligibleSegments = 0;
    int passSegments = 0;

    int localEligible = 0;
    int localPositive = 0;
    double sumLocalImprovement = 0.0;
    double worstLocal = 0.0;
    double bestLocal = 0.0;
    bool localInitialized = false;

    int structuralEligible = 0;
    int structuralPositive = 0;
    double sumStructuralImprovement = 0.0;
    double worstStructural = 0.0;
    double bestStructural = 0.0;
    bool structuralInitialized = false;

    int lastSamples = 0;
    double lastLocalImprovement = 0.0;
    double lastStructuralImprovement = 0.0;
    bool lastLocalValid = false;
    bool lastStructuralValid = false;
    bool lastPass = false;

    for(int s = 0; s < NumberOfSegments; s++)
    {
        int index = StatIndex(p, s);
        SegmentStat patternStat = g_patternStats[index];
        SegmentStat parentStat = g_parentStats[index];

        int localDecisive =
            patternStat.localReversal +
patternStat.localBreakout;
        int parentLocalDecisive =
            parentStat.localReversal + parentStat.localBreakout
;

        int structuralDecisive =
            patternStat.structuralOpposite +
patternStat.structuralSameType;
        int parentStructuralDecisive =
            parentStat.structuralOpposite +
parentStat.structuralSameType;

        double localPct = Percent(patternStat.localReversal,
localDecisive);
        double parentLocalPct =
            Percent(parentStat.localReversal,
parentLocalDecisive);
        double structuralPct =

```

```

        Percent(patternStat.structuralOpposite,
structuralDecisive);
        double parentStructuralPct =
            Percent(parentStat.structuralOpposite,
parentStructuralDecisive);

        double localImprovement = -1.0;
        double structuralImprovement = -1.0;

        if(localPct >= 0.0 && parentLocalPct >= 0.0)
            localImprovement = localPct - parentLocalPct;

        if(structuralPct >= 0.0 && parentStructuralPct >= 0.0)
            structuralImprovement = structuralPct -
parentStructuralPct;

        bool eligibleLocal =
            patternStat.samples >= MinSegmentSamples &&
            localDecisive >= MinLocalDecisiveSamples &&
            parentLocalDecisive >= MinLocalDecisiveSamples;

        bool eligibleStructural =
            patternStat.samples >= MinSegmentSamples &&
            structuralDecisive >= MinStructuralDecisiveSamples
&&
            parentStructuralDecisive >=
MinStructuralDecisiveSamples;

        bool localEdge =
            eligibleLocal &&
            localImprovement >= MinSegmentImprovementPctPoints;

        bool structuralEdge =
            eligibleStructural &&
            structuralImprovement >=
MinSegmentImprovementPctPoints;

        bool segmentEligible = true;
        bool segmentPass = true;

        if(IsLocalEdgeRequired(g_patterns[p]))
        {
            segmentEligible = segmentEligible && eligibleLocal;
            segmentPass = segmentPass && localEdge;
        }

        if(IsStructuralEdgeRequired(g_patterns[p]))
        {
            segmentEligible = segmentEligible &&
eligibleStructural;
            segmentPass = segmentPass && structuralEdge;
        }

```

```

if(segmentEligible)
{
    eligibleSegments++;

    if(segmentPass)
        passSegments++;
}

if(eligibleLocal)
{
    localEligible++;
    sumLocalImprovement += localImprovement;

    if(localImprovement >=
MinSegmentImprovementPctPoints)
        localPositive++;

    if(!localInitialized)
    {
        worstLocal = localImprovement;
        bestLocal = localImprovement;
        localInitialized = true;
    }
    else
    {
        if(localImprovement < worstLocal)
            worstLocal = localImprovement;
        if(localImprovement > bestLocal)
            bestLocal = localImprovement;
    }
}

if(eligibleStructural)
{
    structuralEligible++;
    sumStructuralImprovement += structuralImprovement;

    if(structuralImprovement >=
MinSegmentImprovementPctPoints)
        structuralPositive++;

    if(!structuralInitialized)
    {
        worstStructural = structuralImprovement;
        bestStructural = structuralImprovement;
        structuralInitialized = true;
    }
    else
    {
        if(structuralImprovement < worstStructural)
            worstStructural = structuralImprovement;
        if(structuralImprovement > bestStructural)

```

```

        bestStructural = structuralImprovement;
    }
}

if(s == NumberOfSegments - 1)
{
    lastSamples = patternStat.samples;
    lastLocalImprovement = localImprovement;
    lastStructuralImprovement = structuralImprovement;
    lastLocalValid =
        localPct >= 0.0 && parentLocalPct >= 0.0;
    lastStructuralValid =
        structuralPct >= 0.0 && parentStructuralPct >=
0.0;

    lastPass = segmentEligible && segmentPass;
}
}

double passShare =
    eligibleSegments > 0
    ? 100.0 * passSegments / eligibleSegments
    : -1.0;

double localShare =
    localEligible > 0
    ? 100.0 * localPositive / localEligible
    : -1.0;

double structuralShare =
    structuralEligible > 0
    ? 100.0 * structuralPositive / structuralEligible
    : -1.0;

double avgLocalImprovement =
    localEligible > 0
    ? sumLocalImprovement / localEligible
    : -1.0;

double avgStructuralImprovement =
    structuralEligible > 0
    ? sumStructuralImprovement / structuralEligible
    : -1.0;

string grade;
bool stableAccepted = false;

if(eligibleSegments < MinEligibleSegments)
{
    grade = "INSUFFICIENT_SEGMENTS";
    insufficientCount++;
}
else

```

```

{
    double passShareFraction = passShare / 100.0;

    if(passShareFraction >= StablePassShare &&
        (!RequireLastSegmentPassForStable || lastPass))
    {
        grade = "STABLE";
        stableAccepted = true;
        stableCount++;
    }
    else if(passShareFraction >= PromisingPassShare)
    {
        grade = "PROMISING";
        promisingCount++;
    }
    else
    {
        grade = "UNSTABLE";
        unstableCount++;
    }
}

string line = "";

AppendField(line, IntegerToString(g_patterns[p].rank));
AppendField(line, g_patterns[p].patternID);
AppendField(line, g_patterns[p].selectionReason);
AppendField(line, g_patterns[p].reliabilityTier);
AppendField(line, g_patterns[p].patternType);
AppendField(line, g_patterns[p].ageBasis);
AppendField(line, g_patterns[p].symbol);
AppendField(line, g_patterns[p].timeframe);
AppendField(line, g_patterns[p].zoneType);
AppendField(line, g_patterns[p].ageBucket);
AppendField(line, g_patterns[p].touchBucket);
AppendField(line, g_patterns[p].depthBucket);
AppendField(line, g_patterns[p].approachBucket);
AppendField(line, g_patterns[p].heightBucket);
AppendField(line, g_patterns[p].sessionBucket);
AppendField(line, IntegerToString(g_patterns[p]
.originalSamples));
AppendField(line, DoubleToString(g_patterns[p]
.originalScore, 6));
AppendField(line, IntegerToString(NumberOfSegments));
AppendField(line, IntegerToString(eligibleSegments));
AppendField(line, IntegerToString(passSegments));
AppendField(line, DoubleOrBlank(passShare, 4));
AppendField(line, IntegerToString(localEligible));
AppendField(line, IntegerToString(localPositive));
AppendField(line, DoubleOrBlank(localShare, 4));
AppendField(
    line,

```

```

        SignedDoubleOrBlank(
            avgLocalImprovement,
            4,
            localEligible > 0
        )
    );
    AppendField(line, localInitialized ? DoubleToString(
worstLocal, 4) : "");
    AppendField(line, localInitialized ? DoubleToString(
bestLocal, 4) : "");
    AppendField(line, IntegerToString(structuralEligible));
    AppendField(line, IntegerToString(structuralPositive));
    AppendField(line, DoubleOrBlank(structuralShare, 4));
    AppendField(
        line,
        SignedDoubleOrBlank(
            avgStructuralImprovement,
            4,
            structuralEligible > 0
        )
    );
    AppendField(line, structuralInitialized ? DoubleToString(
worstStructural, 4) : "");
    AppendField(line, structuralInitialized ? DoubleToString(
bestStructural, 4) : "");
    AppendField(line, IntegerToString(lastSamples));
    AppendField(
        line,
        SignedDoubleOrBlank(
            lastLocalImprovement,
            4,
            lastLocalValid
        )
    );
    AppendField(
        line,
        SignedDoubleOrBlank(
            lastStructuralImprovement,
            4,
            lastStructuralValid
        )
    );
    AppendField(line, YesNo(lastPass));
    AppendField(line, grade);
    AppendField(line, YesNo(stableAccepted));

    FileWriteString(handle, line + "\r\n");
}

FileFlush(handle);
FileClose(handle);

```

```

Print("Файл создан: ",
      TerminalInfoString(TERMINFO_DATA_PATH),
      "\\MQL4\\Files\\", OutputWalkForwardSummaryFileName);

Print("Блок 05, итог: STABLE=", stableCount,
      ", PROMISING=", promisingCount,
      ", UNSTABLE=", unstableCount,
      ", INSUFFICIENT=", insufficientCount);

return(true);
}

//+-----+
//| Проверка параметров
//+-----+
bool ValidateInputs()
{
    if(NumberOfSegments < 2 || NumberOfSegments > 50)
    {
        Print("Ошибка: NumberOfSegments должен быть от 2 до 50."
    );
        return(false);
    }

    if(MinSegmentSamples < 1 ||
       MinLocalDecisiveSamples < 1 ||
       MinStructuralDecisiveSamples < 1)
    {
        Print(
"Ошибка: минимальные размеры выборки должны быть больше 0.");
        return(false);
    }

    if(MinEligibleSegments < 1 || MinEligibleSegments >
NumberOfSegments)
    {
        Print(
"Ошибка: MinEligibleSegments вне допустимого диапазона.");
        return(false);
    }

    if(StablePassShare < 0.0 || StablePassShare > 1.0 ||
       PromisingPassShare < 0.0 || PromisingPassShare > 1.0 ||
       PromisingPassShare > StablePassShare)
    {
        Print(
"Ошибка: неверные значения StablePassShare/PromisingPassShare."
    );
        return(false);
    }
}

```

```

if(AsiaEndHour < 0 || AsiaEndHour > 24 ||
    LondonEndHour < 0 || LondonEndHour > 24 ||
    NewYorkEndHour < 0 || NewYorkEndHour > 24 ||
    AsiaEndHour > LondonEndHour ||
    LondonEndHour > NewYorkEndHour)
{
    Print("Ошибка: неверные границы торговых сессий.");
    return(false);
}

return(true);
}

//+-----

//| Точка входа скрипта

//+-----
void OnStart()
{
    Print(
"=====
");
    Print(
"RectangleZoneAnalyzer_05: начало хронологической проверки");
    Print(
"=====
");

    if(!ValidateInputs())
        return;

    if(!LoadPatterns())
        return;

    if(!ScanTouchTimeRange())
        return;

    if(!BuildSegments())
        return;

    for(int s = 0; s < NumberOfSegments; s++)
    {
        Print("Сегмент ", s + 1, ": ",
            DateTimeText(g_segmentStart[s]), " - ",
            DateTimeText(g_segmentEnd[s]));
    }

    if(!AllocateStatistics())
        return;

    if(!AggregateSegments())
        return;

```

```
if(!SaveWalkForwardDetails())
    return;

if(!SaveWalkForwardSummary())
    return;

Print(
"=====
");
Print("RectangleZoneAnalyzer_05: завершено успешно");
Print(
"=====
");
}

//+-----
```